

Multi-Source RAG Pipeline using LangChain, ChromaDB, and Hugging Face Embeddings

Problem Statement

The rapid growth of unstructured data such as PDFs, websites, research papers, and text files creates a major challenge in retrieving accurate and meaningful information efficiently. Traditional keyword-based search systems often fail to understand the semantic meaning of user queries, leading to irrelevant or incomplete results.

Objective

This project builds a RAG (Retrieval-Augmented Generation) Pipeline using LangChain, Chroma, and Hugging Face embeddings in Google Colab to retrieve relevant information from PDFs, websites, and text files. The system converts unstructured data into embeddings, stores them in a vector database, and performs semantic similarity search to provide accurate and context-aware responses to user queries.

Steps Used in the Project

- Step 1: Install Required Libraries
- Step 2: Load Text File Data
- Step 3: Load Website Data
- Step 4: Load PDF Documents
- Step 5: Split Documents into Chunks
- Step 6: Generate Embeddings and Store Embeddings in Vector Database
- Step 7: Perform Similarity Search

Step 1: Install Required Libraries

```
# !pip install pypdf
# !pip install -q langchain langchain-community langchain-text-splitters
# !pip install -q langchain-huggingface sentence-transformers chromadb
```

Collecting pypdf
 Downloading pypdf-6.11.0-py3-none-any.whl.metadata (7.2 kB)
 Downloading pypdf-6.11.0-py3-none-any.whl (338 kB)
 _____ 338.8/338.8 kB 2.4 MB/s eta 0:00:00

Installing collected packages: pypdf
Successfully installed pypdf-6.11.0

_____ 2.5/2.5 MB 9.1 MB/s eta 0:00:00
_____ 1.0/1.0 MB 56.0 MB/s eta 0:00:00
_____ 548.1/548.1 kB 36.5 MB/s eta 0:00:00
_____ 73.1/73.1 kB 7.4 MB/s eta 0:00:00
_____ 51.0/51.0 kB 4.7 MB/s eta 0:00:00

ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the source of the following dependency conflicts.
google-colab 1.0.0 requires requests==2.32.4, but you have requests 2.34.2 which is incompatible.

_____ 52.0/52.0 kB 924.0 kB/s eta 0:00:00
_____ 23.3/23.3 MB 39.4 MB/s eta 0:00:00
_____ 278.2/278.2 kB 20.5 MB/s eta 0:00:00
_____ 2.0/2.0 MB 70.6 MB/s eta 0:00:00
_____ 18.2/18.2 MB 42.3 MB/s eta 0:00:00
_____ 72.1/72.1 kB 6.1 MB/s eta 0:00:00
_____ 180.2/180.2 kB 17.8 MB/s eta 0:00:00
_____ 69.0/69.0 kB 5.8 MB/s eta 0:00:00
_____ 231.6/231.6 kB 22.0 MB/s eta 0:00:00
_____ 71.6/71.6 kB 7.2 MB/s eta 0:00:00
_____ 60.6/60.6 kB 6.1 MB/s eta 0:00:00

ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the source of the following dependency conflicts.
google-adk 1.28.0 requires opentelemetry-api<1.39.0,>=1.36.0, but you have opentelemetry-api 1.41.1 which is incompatible.
google-adk 1.28.0 requires opentelemetry-sdk<1.39.0,>=1.36.0, but you have opentelemetry-sdk 1.41.1 which is incompatible.
opentelemetry-exporter-gcp-logging 1.11.0a0 requires opentelemetry-sdk<1.39.0,>=1.35.0, but you have opentelemetry-sdk 1.41.1 which is incompatible.

opentelemetry-exporter-otlp-proto-http 1.38.0 requires opentelemetry-exporter-otlp-proto-common==1.38.0, but you have opentelemetry-exp
opentelemetry-exporter-otlp-proto-http 1.38.0 requires opentelemetry-proto==1.38.0, but you have opentelemetry-proto 1.41.1 which is in
opentelemetry-exporter-otlp-proto-http 1.38.0 requires opentelemetry-sdk~=1.38.0, but you have opentelemetry-sdk 1.41.1 which is incomp

Step 2: Load Text File Data

```
# Upload the file to Colab or connect your Google Drive
from google.colab import files

# If using Google Drive, mount it
#from google.colab import drive
#drive.mount('/content/drive')

# Adjust the file path as needed
#file_path = '/content/drive/MyDrive/RAG-Pipeline/files/speech.txt'

file_path = 'speech.txt' # The file will be uploaded to the current session directory

# Text Loader
from langchain_community.document_loaders import TextLoader
loader = TextLoader(file_path)
text_doc = loader.load()
print(text_doc)

[Document(metadata={'source': 'speech.txt'}, page_content='The world must be made safe for democracy. Its peace must be planted upon th
```

Step 3: Load Website Data

```
# Web Based Loader
from langchain_community.document_loaders import WebBaseLoader
import bs4

# load, chunk and index the content of the html page

loader = WebBaseLoader(web_paths=('https://lilianweng.github.io/posts/2023-06-23-agent/'),
                        bs_kwargs=dict(parse_only=bs4.SoupStrainer(
                            class_=('post-title', 'post-content', 'post-header')
                        )))

web_loader = loader.load()
web_loader
```

larger knowledge pool, their representation power is not as powerful as full attention.

Challenges in long-term planning and task decomposition: Planning over a lengthy history and effectively exploring the solution space remain challenging. LLMs struggle to adjust plans when faced with unexpected errors, making them less robust compared to humans who learn from trial and error.

Reliability of natural language interface: Current agent system relies on natural language as an interface between LLMs and external components such as memory and tools. However, the reliability of model outputs is questionable, as LLMs may make formatting errors and occasionally exhibit rebellious behavior (e.g. refuse to follow an instruction). Consequently, much of the agent demo code focuses on parsing model output.

Citation#Cited as: Weng, Lilian. (Jun 2023). "LLM-powered Autonomous Agents". Lil'Log. <https://lilianweng.github.io/posts/2023-06-23-agent/>.

title = "LLM-powered Autonomous Agents", author = "Weng, Lilian", journal = "lilianweng.github.io", year = "2023", month = "Jun", url = "https://lilianweng.github.io/posts/2023-06-23-agent/"

References#

- Wei et al. "Chain of thought prompting elicits reasoning in large language models." NeurIPS 2022
- Yao et al. "Tree of Thoughts: Deliberate Problem Solving with Large Language Models." arXiv preprint arXiv:2305.10601 (2023).
- Liu et al. "Chain of Hindsight Aligns Language Models with Feedback" arXiv preprint arXiv:2302.02676 (2023).
- Liu et al. "LLM+P: Empowering Large Language Models with Optimal Planning Proficiency" arXiv preprint arXiv:2304.11477 (2023).
- Yao et al. "ReAct: Synergizing reasoning and acting in language models." ICLR 2023.
- Google Blog. "Announcing ScaNN: Efficient Vector Similarity Search" July 28, 2020. <https://chat.openai.com/share/46ff149e-a4c7-4dd7-a800-fc4a642ea389>
- Shinn & Labash. "Reflexion: an autonomous agent with dynamic memory and self-reflection" arXiv preprint arXiv:2303.11366 (2023).
- Laskin et al. "In-context Reinforcement Learning with Algorithm Distillation" ICLR 2023.
- Karpas et al. "MRKL Systems A modular, neuro-symbolic architecture that combines large language models, external knowledge sources and discrete reasoning." arXiv preprint arXiv:2205.00445 (2022).
- Nakano et al. "Webgpt: Browser-assisted question-answering with human feedback." arXiv preprint arXiv:2112.09332 (2021).
- Parisi et al. "TALM: Tool Augmented Language Models" arXiv preprint arXiv:2302.04761 (2023).
- Weaviate Blog. Why is Vector Search so fast? Sep 13, 2022.
- Li et al. "API-Bank: A Benchmark for Tool-Augmented LLMs" arXiv preprint arXiv:2304.08244 (2023).
- Shen et al. "HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in HuggingFace" arXiv preprint arXiv:2303.17580 (2023).
- Bran et al. "ChemCrow: Augmenting large-language models with chemistry tools." arXiv preprint arXiv:2304.05376 (2023).
- Boiko et al. "Emergent autonomous scientific research capabilities of large language models." arXiv preprint arXiv:2304.05332 (2023).
- Joon Sung Park, et al. "Generative Agents: Interactive Simulacra of Human Behavior." arXiv preprint arXiv:2304.03442 (2023).
- AutoGPT. <https://github.com/Significant-Gravitas/Auto-GPT>
- GPT-Engineer. <https://github.com/AntonOvika/gpt-engineer>

Step 4: Load PDF Documents

```
# PDF Reader
from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader('attention.pdf')

pdf_loader = loader.load()
pdf_loader
```

apparently involved in anaphora resolution. Top: Full attentions for head 5. Bottom: Isolated attentions from just the word 'its' for attention heads 5 and 6. Note that the attentions are very sharp for this word.

Document(metadata={'producer': 'pdfTeX-1.40.25', 'creator': 'LaTeX with hyperref', 'creationdate': '2023-08-03T00:07:29+00:00', 'author': '', 'keywords': '', 'moddate': '2023-08-03T00:07:29+00:00', 'ptex.fullbanner': 'This is pdfTeX, Version 3.141592653-2.6-1.40.25 (TeX Live 2023) kpathsea version 6.3.5', 'subject': '', 'title': '', 'trapped': '/False', 'source': 'attention.pdf', 'total_pages': 15, 'page': 14, 'page_label': '15'}, page_content='Input-Input Layer5\nThe\nLaw\nwill\nnever\nbe\nperfect\n,\nbut\nits\napplication\nshould\nbe\njust\n-\nthis\nis\nwhat\nwe\nare\nmissing\n,\nin\nmy\nopinion\n.\n<EOS>\n<pad>\nInput-Input Layer5\nThe\nLaw\nwill\nnever\nbe\nperfect\n,\nbut\nits\napplication\nshould\nbe\njust\n-\nthis\nis\nwhat\nwe\nare\nmissing\n,\nin\nmy\nopinion\n.\n<EOS>\n<pad>\nThe\nLaw\nwill\nnever\nbe\nperfect\n,\nbut\nits\napplication\nthis\nis\nwhat\nwe\nare\nmissing\n,\nin\nmy\nopinion\n.\n<EOS>\n<pad>\nFigure 5: Many of the attention heads exhibit behaviour that seems related to the structure of the sentence. We give two such examples above, from two different heads from the encoder self-attention\nat layer 5 of 6. The heads clearly learned to perform different tasks.\n15'})]

Step 5: Split Documents into Chunks

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
documents = text_splitter.split_documents(pdf_loader)
documents[:5]
```

[Document(metadata={'producer': 'pdfTeX-1.40.25', 'creator': 'LaTeX with hyperref', 'creationdate': '2023-08-03T00:07:29+00:00', 'author': '', 'keywords': '', 'moddate': '2023-08-03T00:07:29+00:00', 'ptex.fullbanner': 'This is pdfTeX, Version 3.141592653-2.6-1.40.25 (TeX Live 2023) kpathsea version 6.3.5', 'subject': '', 'title': '', 'trapped': '/False', 'source': 'attention.pdf', 'total_pages': 15, 'page': 0, 'page_label': '1'}, page_content='Provided proper attribution is provided, Google hereby grants permission to reproduce the tables and figures in this paper solely for use in journalistic or scholarly works.\nAttention Is All You Need\nAshish Vaswani*\nGoogle Brain\nnavaswani@google.com\nNoam Shazeer*\nGoogle Brain\nnoam@google.com\nNiki Parmar*\nGoogle Research\nnnikip@google.com\nJakob Uszkoreit*\nGoogle Research\nnusz@google.com\nLlion Jones*\nGoogle Research\nllion@google.com\nAidan N. Gomez* \nUniversity of Toronto\nnaidan@cs.toronto.edu\ntukasz Kaiser *\nGoogle Brain\nnlukaszkaizer@google.com\nIllia Polosukhin* \nillia.polosukhin@gmail.com\nAbstract\nThe dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions'),

Document(metadata={'producer': 'pdfTeX-1.40.25', 'creator': 'LaTeX with hyperref', 'creationdate': '2023-08-03T00:07:29+00:00', 'author': '', 'keywords': '', 'moddate': '2023-08-03T00:07:29+00:00', 'ptex.fullbanner': 'This is pdfTeX, Version 3.141592653-2.6-1.40.25 (TeX Live 2023) kpathsea version 6.3.5', 'subject': '', 'title': '', 'trapped': '/False', 'source': 'attention.pdf', 'total_pages': 15, 'page': 0, 'page_label': '1'}, page_content='mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.'),

Document(metadata={'producer': 'pdfTeX-1.40.25', 'creator': 'LaTeX with hyperref', 'creationdate': '2023-08-03T00:07:29+00:00', 'author': '', 'keywords': '', 'moddate': '2023-08-03T00:07:29+00:00', 'ptex.fullbanner': 'This is pdfTeX, Version 3.141592653-2.6-1.40.25 (TeX Live 2023) kpathsea version 6.3.5', 'subject': '', 'title': '', 'trapped': '/False', 'source': 'attention.pdf', 'total_pages': 15, 'page': 0, 'page_label': '1'}, page_content='best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.\n*Equal contribution. Listing order is random. Jakob proposed replacing RNNs with self-attention and started the effort to evaluate this idea. Ashish, with Illia, designed and implemented the first Transformer models and has been crucially involved in every aspect of this work. Noam proposed scaled dot-product attention, multi-head attention and the parameter-free position representation and became the other person involved in nearly every detail. Niki designed, implemented, tuned and evaluated countless model variants in our original codebase and tensor2tensor. Llion also experimented with novel model variants, was responsible for our initial codebase, and inefficient inference and visualizations. Lukasz and Aidan spent countless long days designing various parts of and'),

Document(metadata={'producer': 'pdfTeX-1.40.25', 'creator': 'LaTeX with hyperref', 'creationdate': '2023-08-03T00:07:29+00:00', 'author': '', 'keywords': '', 'moddate': '2023-08-03T00:07:29+00:00', 'ptex.fullbanner': 'This is pdfTeX, Version 3.141592653-2.6-1.40.25 (TeX Live 2023) kpathsea version 6.3.5', 'subject': '', 'title': '', 'trapped': '/False', 'source': 'attention.pdf', 'total_pages': 15, 'page': 0, 'page_label': '1'}, page_content='efficient inference and visualizations. Lukasz and Aidan spent countless long days designing various parts of and implementing tensor2tensor, replacing our earlier codebase, greatly improving results and massively accelerating our research.\nWork performed while at Google Brain.\nWork performed while at Google Research.\n31st Conference on Neural Information Processing Systems (NIPS 2017), Long Beach, CA, USA.\narXiv:1706.03762v7 [cs.CL] 2 Aug 2023'),

Document(metadata={'producer': 'pdfTeX-1.40.25', 'creator': 'LaTeX with hyperref', 'creationdate': '2023-08-03T00:07:29+00:00', 'author': '', 'keywords': '', 'moddate': '2023-08-03T00:07:29+00:00', 'ptex.fullbanner': 'This is pdfTeX, Version 3.141592653-2.6-1.40.25 (TeX Live 2023) kpathsea version 6.3.5', 'subject': '', 'title': '', 'trapped': '/False', 'source': 'attention.pdf', 'total_pages': 15, 'page': 1, 'page_label': '2'}, page_content='1 Introduction\nRecurrent neural networks, long short-term memory [13] and gated recurrent [7] neural networks\nin particular, have been firmly established as state of the art approaches in sequence modeling and transduction problems such as language modeling and machine translation [35, 2, 5]. Numerous efforts have since continued to push the boundaries of recurrent language models and encoder-decoder architectures [38, 24, 15].\nRecurrent models typically factor computation along the symbol positions of the input and output sequences. Aligning the positions to steps in computation time, they generate a sequence of hidden states h_t , as a function of the previous hidden state h_{t-1} and the input for position t . This inherently sequential nature precludes parallelization within training examples, which becomes critical at longer sequence lengths, as memory constraints limit batching across examples. Recent work has achieved')]

Step 6: Generate Embeddings and Store Embeddings in Vector Database

```
# Embedding
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_community.vectorstores import Chroma

# Initialize the HuggingFaceEmbeddings with the instructor model
embeddings = HuggingFaceEmbeddings(model_name="hkunlp/instructor-xl")

# Create vectorstore for the first 10 documents using Chroma
db = Chroma.from_documents(documents[:10], embedding=embeddings)

print("Indexed Documents Successfully!")
```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF\_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (<https://huggingface.co/settings/tokens>), set it as secret in your Colab secrets.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.

```
warnings.warn(
modules.json: 100% 461/461 [00:00<00:00, 25.2kB/s]

config_sentence_transformers.json: 100% 122/122 [00:00<00:00, 9.70kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster download
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable hi
README.md: 66.3k/? [00:00<00:00, 970kB/s]

sentence_bert_config.json: 100% 53.0/53.0 [00:00<00:00, 3.93kB/s]

config.json: 1.52k/? [00:00<00:00, 17.1kB/s]

pytorch_model.bin: 100% 4.96G/4.96G [00:47<00:00, 189MB/s]

Loading weights: 100% 196/196 [00:00<00:00, 580.94it/s, Materializing param=shared.weight]
The tied weights mapping and config for this model specifies to tie shared.weight to encoder.embed_tokens.weight, but both are present
tokenizer_config.json: 2.40k/? [00:00<00:00, 154kB/s]

model.safetensors: 100% 4.96G/4.96G [00:50<00:00, 184MB/s]

spiece.model: 100% 792k/792k [00:02<00:00, 3.95MB/s]

tokenizer.json: 2.42M/? [00:00<00:00, 31.2MB/s]

special_tokens_map.json: 2.20k/? [00:00<00:00, 16.0kB/s]

config.json: 100% 270/270 [00:00<00:00, 682B/s]

config.json: 100% 116/116 [00:00<00:00, 2.95kB/s]

2_Dense/pytorch_model.bin: 100% 3.15M/3.15M [00:03<00:00, 15.9MB/s]

Indexed Documents Successfully!
```

Step 7: Perform Similarity Search

```
query = 'Who are the authors of the attention is all you need research paper'
result = db.similarity_search(query)
result[0].page_content
```

'Provided proper attribution is provided, Google hereby grants permission to\nreproduce the tables and figures in this paper solely fo
r use in journalistic or\nscholarly works.\nAttention Is All You Need\nAshish Vaswani*\nGoogle Brain\nnavaswani@google.com\nNoam Shazeer*\nGoogle Brain\nnoam@google.com\nNiki Parmar*\nGoogle Research\nnnikip@google.com\nJakob Uszkoreit*\nGoogle Research\nnusz@google.com\nLlion Jones*\nGoogle Research\nllion@google.com\nAidan N. Gomez* †\nUniversity of Toronto\nnaidan@cs.toronto.edu\nŁukasz Kaiser *\nGoogle Brain\nlukaszkaiser@google.com\nIllia Polosukhin* †\nillia.polosukhin@gmail.com\nAbstract\nThe dominant sequence transduction models are based on complex recurrent or\nconvolutional neural networks that include an encoder and a decoder. The best\nperforming models al
so connect the encoder and decoder through an attention\nmechanism. We propose a new simple network architecture, the Transformer,\nba
sed solely on attention mechanisms, dispensing with recurren...'

```
# Assuming result[0].page_content is the raw text you're working with
result_content = result[0].page_content

# Print the content as is, preserving the newlines
print(result_content)
```

Provided proper attribution is provided, Google hereby grants permission to reproduce the tables and figures in this paper solely for use in journalistic or scholarly works.

Attention Is All You Need

Ashish Vaswani*

Google Brain

avaswani@google.com

Noam Shazeer*

Google Brain

noam@google.com

Niki Parmar*

Google Research

nikip@google.com

Jakob Uszkoreit*

Google Research

usz@google.com

Llion Jones*

Google Research

llion@google.com

Aidan N. Gomez* †

University of Toronto

aidan@cs.toronto.edu

Łukasz Kaiser *

Google Brain

lukaszkaiser@google.com

Illia Polosukhin* ‡

illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.